

# PhantomCortex Static Model

September 2026 Retrain: Findings, Fixes, and Honest Thresholds

A plain-language and technical account of the crash investigation, memory-lifecycle fix, split-discipline retrain, honest threshold derivation, one-time test read, and promotion carried out on the Cortex-Static model.

Repository: cortex-ml | Prepared: September 22, 2026  
Source commits: 8192452, 48f55f0, 9030d6a, 438d936

# Table of Contents

|                                          |           |
|------------------------------------------|-----------|
| <b>1. Executive Summary</b>              | <b>3</b>  |
| <b>2. The Problem and the System</b>     | <b>4</b>  |
| <b>3. How a Decision Gets Made</b>       | <b>6</b>  |
| <b>4. Project History</b>                | <b>8</b>  |
| <b>5. This Session's Work, In Detail</b> | <b>9</b>  |
| a. The crash . . . . .                   | 9         |
| b. The investigation . . . . .           | 9         |
| c. The fix . . . . .                     | 9         |
| d. The real training run . . . . .       | 10        |
| e. The honest thresholds . . . . .       | 10        |
| f. The one-time test . . . . .           | 10        |
| g. Promotion . . . . .                   | 10        |
| <b>6. Test Results — Old vs New</b>      | <b>12</b> |
| <b>7. What's Better Now</b>              | <b>13</b> |
| <b>8. What's Still a Problem</b>         | <b>14</b> |
| <b>9. What Changed, Concretely</b>       | <b>15</b> |
| <b>10. What's Left</b>                   | <b>17</b> |
| <b>Closing</b>                           | <b>18</b> |

# 1. Executive Summary

PhantomCortex is an automated system that decides, for every file that shows up on a protected Windows machine, whether it looks safe, suspicious, or dangerous. One of its five independent judges — Cortex-Static, which reads a file's structure without running it — had a quietly serious problem: the score cut-offs that decide "let it through" versus "block it" had been picked by looking at the exact same data later used to grade the model, the equivalent of a teacher handing out the answer key before grading the exam. That made the model's report card look better than it actually was, and it sat alongside a separate, unrelated crash bug that made the model impossible to retrain on this machine at all, because the retraining process ran out of memory every time.

This session fixed both problems. The memory crash was traced to a single lingering reference to a large array of training data that Python could not release, fixed, and proven fixed by measuring an 18-gigabyte memory drop exactly where none had been happening before. With that fixed, the static model was fully retrained from scratch, its score cut-offs were derived honestly on data the model never learned from, written down and locked in before anyone was allowed to look at the final exam, and then — and only then — graded once on that final exam. The new numbers are close to the old ones and, unlike the old ones, can actually be trusted.

What is left: the new model was promoted into production with full rollback available, but a known, separate false-positive pattern on certain legitimate files (large PyInstaller- packaged programs) was not fixed by this work, and the safety brake that already exists to protect against it — described in Section 3 — stays firmly in place. The follow-on work to close that gap has not started.

## 2. The Problem and the System

PhantomCortex looks at a file the moment it appears and asks, in effect, "is this safe to run?" It does not rely on a single test to answer that question. Instead it runs five independent checks and only then combines their answers:

- **Cortex-Static** looks at the file itself while it sits still on disk — its internal structure, the functions it imports, the text strings buried inside it — without ever running it.
- **Cortex-Behavioral** looks at the sequence of system calls a program makes once it is actually running, the way a security guard watches what someone does rather than just what they look like.
- **Cortex-Memory** looks at a snapshot of a running computer's memory for signs of a program hiding inside another one, which catches attacks that never write a file to disk at all.
- **Cortex-Network** looks at a computer's network traffic patterns for the fingerprints of a program "phoning home" or trying to steal data.
- **Cortex-Emulation** runs a file inside a disposable, sandboxed copy of Windows and watches what it does, though its results are currently only recorded for later study, not used in any live decision (Section 7 explains why).

The reason for five separate checks instead of one clever check is straightforward: each one has a different blind spot, and a file that fools one check will usually not fool another. A program can be well-behaved on disk and only reveal itself once it starts making network connections; a fileless attack that never touches disk at all is invisible to Cortex-Static by definition but is exactly what Cortex-Memory exists to catch. No single signal is trusted to make a life-or-death decision alone except in the two cases explained next.

## How PhantomCortex Decides: A File's Journey

A plain-English walk-through, in the order every scan actually happens

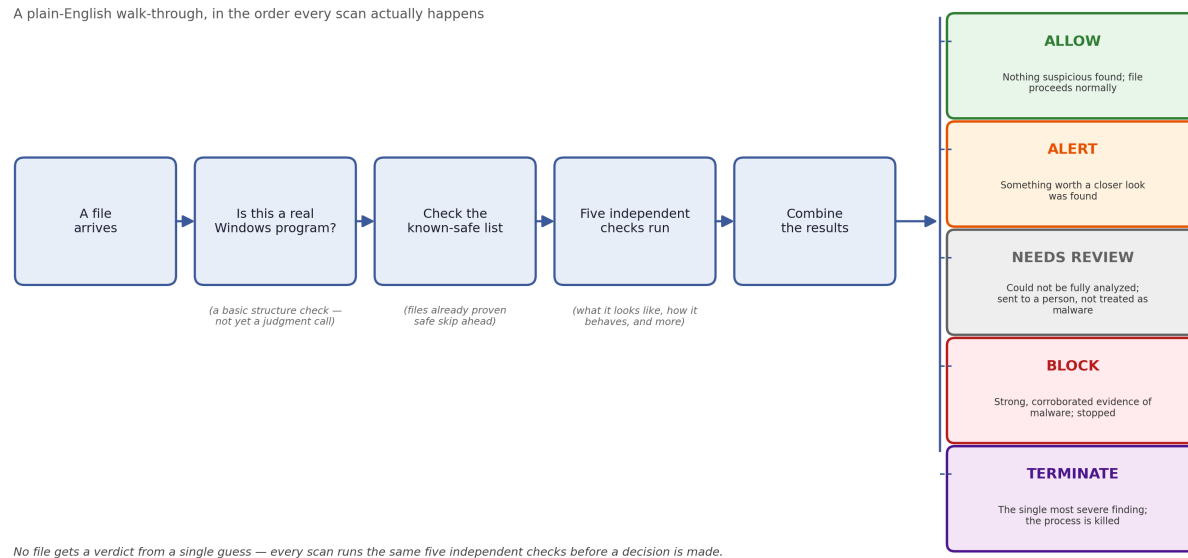


Figure 1. A file's journey through PhantomCortex, in plain language.

### 3. How a Decision Gets Made

Follow one file through the system. It arrives, and the system first asks a purely mechanical question: is this actually a well-formed Windows program at all, based on its internal file structure? This is not a judgment call — a corrupted or non-Windows file fails here regardless of content. If it passes, the system checks a known-safe list: if the file's exact fingerprint (a "SHA-256 hash," a unique digital fingerprint computed from the file's contents) matches a file already independently verified safe, or if the file carries a cryptographically verified digital signature chaining to a trusted publisher, it is allowed through immediately and none of Cortex-Static's own judgment is needed for it.

Otherwise, the file's characteristics — file size, the pattern of "randomness" in its bytes (a data-compression concept called "entropy" that packed or encrypted malware tends to maximize), which system functions it imports, the text it contains, and roughly two thousand other measurements — are converted into a single, self-consistent format the model understands, called a "feature vector." That vector is handed to Cortex-Static's machine-learning model, an ensemble of ~3,000 decision trees called a "LightGBM" model.

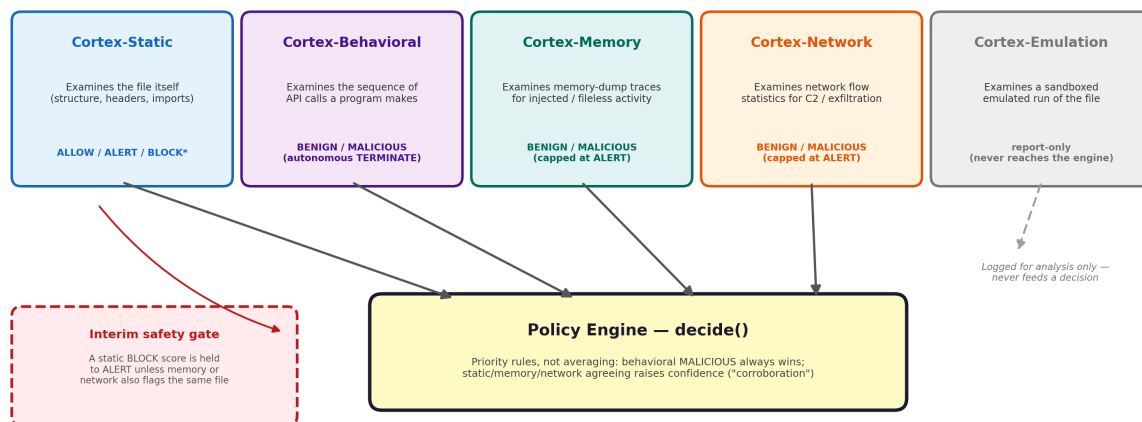
The raw output of that model is not, by itself, a probability anyone can trust: it is a tree ensemble's internal score, which can be badly overconfident (a model might say "99.99% malicious" when the honest figure, on data it has never seen, is closer to 90%). A second, much simpler step called "calibration" fixes this: a small correction curve, fit specifically on data the tree model never trained on, converts the raw score into an honest probability between 0% ("definitely safe, as far as this signal can tell") and 100% ("definitely malicious").

Even an honest probability is not a decision by itself — someone has to decide how confident is confident enough. That is what a "threshold" is: a specific cut line, chosen in advance, such that any score below it means ALLOW, any score above a second, higher line means an escalation, and anything in between means "worth a second look" (ALERT). Choosing that line is itself a trade-off between missing real malware and wrongly flagging safe files, and Section 4 explains exactly why the way this project used to choose that line was not trustworthy.

Cortex-Static's most severe verdict, BLOCK, comes with a deliberate exception worth explaining plainly: **a static BLOCK score alone does not stop a file.** It is held back to the lesser ALERT level unless a second, independent signal — Cortex-Memory or Cortex-Network — also flags the same file. Think of it as a safety brake on a single sensor: Cortex-Static's judgment, on its own, has a known false-positive pattern on a specific class of legitimate file (Section 8 names the actual files), so the system deliberately declines to let it act alone at the most severe level, even though the code that would let it do so still exists and works — it is a policy decision, not a technical limitation. Removing that brake requires proving, with real-world evidence, that this false-positive pattern is fixed, which has not happened yet.

## System Architecture: Five Signals, One Policy Engine

Scores are never averaged. Each signal reaches its own verdict; decide() combines them by fixed priority rules.



\* Static's BLOCK is the only autonomous verdict that is deliberately held back — see the interim safety gate above. Behavioral's MALICIOUS verdict is the sole autonomous route to TERMINATE. Memory and Network can only ever reach ALERT on their own.

Figure 2. The five signals and the central policy engine.

## 4. Project History

PhantomCortex's static model has passed through three engineering phases, in three separate code repositories: `malware-ml` (the original research project), `cortex-endpoint` (a packaged Windows application built from that research), and `cortex-ml` (an independent, from-scratch rebuild — the repository this report covers — which reproduced and then extended the original two-signal design into today's five-signal system).

Before this session, the static model in `cortex-ml` carried two documented, related problems, discovered during a large real-file validation round on 2026-08-28:

**Calibration saturation.** The honesty-correction step described in Section 3 had been fit incorrectly — on the model's already-compressed probability output rather than on its raw, uncompressed score, which is what the standard technique actually calls for. The practical effect was that the correction curve became an almost on/off switch: nearly every file landed at either "essentially 0% malicious" or "essentially 100% malicious," with almost nothing in between. That is dangerous specifically because it means the model has almost no ability to distinguish "very likely safe" from "genuinely uncertain" once a file's raw score crosses a certain point — real evidence showed a confirmed-safe file outranking real malware samples once this compression took hold.

**The feature-fidelity gap.** Independently, the way this project's own code reads a file's raw characteristics into that ~2,568-measurement vector had small but real mismatches against the reference implementation the model's training data was built with. For a specific pattern of legitimate program — large, PyInstaller-packaged Windows executables — those small mismatches were shown to inflate the model's raw score before calibration ever touched it.

Separately, and more directly relevant to this session, the project's own written threshold-derivation history documented an even more fundamental issue: **the ALLOW and BLOCK cut-lines currently in production had been chosen by looking directly at the exact data later used to report the model's accuracy**. This is the software equivalent of a teacher handing a student the answer key before grading their exam and then reporting the resulting perfect score as evidence the student is well-prepared. The recorded numbers for those old thresholds' test-set false-positive rates matched their targets almost exactly (0.009999 against a target of 0.01, for instance) — which sounds precise, but that suspicious exactness is itself the tell: it means those numbers were never really being tested against unseen data at all, they were being fit to it. Fixing this required a full retrain with a properly separated evaluation process, which is the work this session performed and this report documents.

## 5. This Session's Work, In Detail

### a. The crash

Retraining the static model on the full ~2.34-million-row training dataset (2,568 measurements per file) repeatedly crashed the training machine by exhausting its available memory during a step called "Dataset construction" — the process of converting the raw data into the internal binned format the tree-building algorithm actually uses. Three real attempts at a fix all failed at this step, not during the tree-building itself, with one measured run climbing from about 19 to over 36 gigabytes of memory used in just over two minutes before it had to be stopped.

### b. The investigation

A first, plausible-sounding fix was tried and measured before anything else: reducing an internal setting called `max_bin`, which controls how finely the tree-building algorithm buckets each measurement's possible values. This looked promising on paper — fewer buckets should mean less memory. **This lead turned out to be measuring the wrong thing entirely.** The test that produced encouraging-looking numbers had, by accident, generated its practice data in a format that used twice the memory it should have (a floating-point precision mismatch), so the "crash" it was actually measuring was caused by that test's own data-generation step, not by the real training pipeline at all. Once this was caught and corrected, the `max_bin` question was left genuinely unresolved rather than falsely "solved" — a good example of why a promising-looking number needs to be checked against what it actually measured before being trusted.

The real cause was found by direct measurement, not guesswork: a careful read of the training code showed that a large chunk of training data — an array of roughly 18 gigabytes — was being kept alive in memory by a lingering reference the calling code held onto, even after the function that was supposed to be done with that data had finished using it and tried to release its own copy. In Python, deleting one function's local reference to a piece of data does nothing if another part of the program still holds its own separate reference to the same data — and that is exactly what was happening here. The fix that had already been attempted (telling the inner function to delete its own copy) was therefore a no-op the whole time, because the outer code's reference kept the data alive regardless.

### c. The fix

The fix restructures how the large training arrays are handed from the outer training script to the inner training function: instead of passing the arrays directly (which keeps a lingering reference alive in the calling code for the entire duration of the call), the outer code now packages them into a temporary container and hands over that container, which the inner function then empties out one piece at a time as it uses each piece — genuinely dropping the last remaining reference to each array the moment it is no longer needed, rather than only appearing to.

This was proven, not assumed. A direct, controlled test confirmed that the previous calling pattern truly did keep the data alive with zero memory recovered when it should have been released, while the new pattern recovered memory that matched, almost exactly, the expected size of the freed array. At full production scale, immediately after the first several-gigabyte training array was supposed to be released, memory usage dropped by 17.91 gigabytes — matching the expected size of that exact array (18,340 megabytes) to

within two-hundredths of one percent. That is about as close to a smoking-gun confirmation as a memory fix can get.

#### **d. The real training run**

With the fix verified at smaller scale and then in a full-scale rehearsal that completed without incident, the real training run was launched: the full production configuration, 3,000 rounds of tree-building, on the complete real training split. It ran for approximately 100 minutes (6,040.8 seconds) and completed cleanly with no memory warnings at any point during the run. The resulting model reached a validation accuracy score (explained below) of 0.999283 — an "AUC," or Area Under the (ROC) Curve, a standard single-number summary of how well a model separates safe files from malicious ones, running from 0.5 (no better than a coin flip) to 1.0 (perfect separation).

#### **e. The honest thresholds**

With a freshly trained model in hand, the ALLOW and BLOCK cut-lines were derived the way they always should have been: exclusively on a dedicated slice of data — called "cal," for calibration — that had been held out of both the tree-building process and the early-stopping decision that chose how many rounds of trees to build. Crucially, this derivation never opened the final exam (the "test" data) at all. A menu of candidate thresholds at several different false-positive-rate targets was produced, complete with statistically rigorous confidence intervals, and two were chosen from that menu.

Those two chosen numbers, and the exact model they belonged to (identified by a cryptographic fingerprint of the trained model file), were then written down in a dated, timestamped document — before the final exam was ever opened. That document also committed, in advance, to the specific pass/fail bar the final result would be judged against (a validation-accuracy floor) and to a rule that no threshold could be changed in response to whatever the final exam showed. This is the software equivalent of a scientist publicly pre-registering a hypothesis before running the experiment that will test it — it makes it impossible to quietly move the goalposts after seeing the answer.

#### **f. The one-time test**

Only after all of the above was locked in was the held-out test data opened — for the first and only time in this entire process. The result: a test accuracy (AUC) of 0.998778, comfortably clearing the pre-committed bar of 0.9985, and closely matching the advance prediction that had been written down beforehand. The detailed pass/fail numbers are in Section 6.

#### **g. Promotion**

The old model and its configuration were backed up in full before anything was replaced, with a rollback script tested in a safe "dry run" mode immediately beforehand to confirm it would actually work if needed. The new model, its calibration correction, and its two threshold values then replaced the old ones in the live configuration, with each file's cryptographic fingerprint re-verified after copying to confirm nothing was corrupted in transit. A one-command rollback remains ready and pre-tested.

## The Static Model's Training Pipeline (September 2026 Retrain)

One split of raw data is set aside and touched exactly once, at the very end

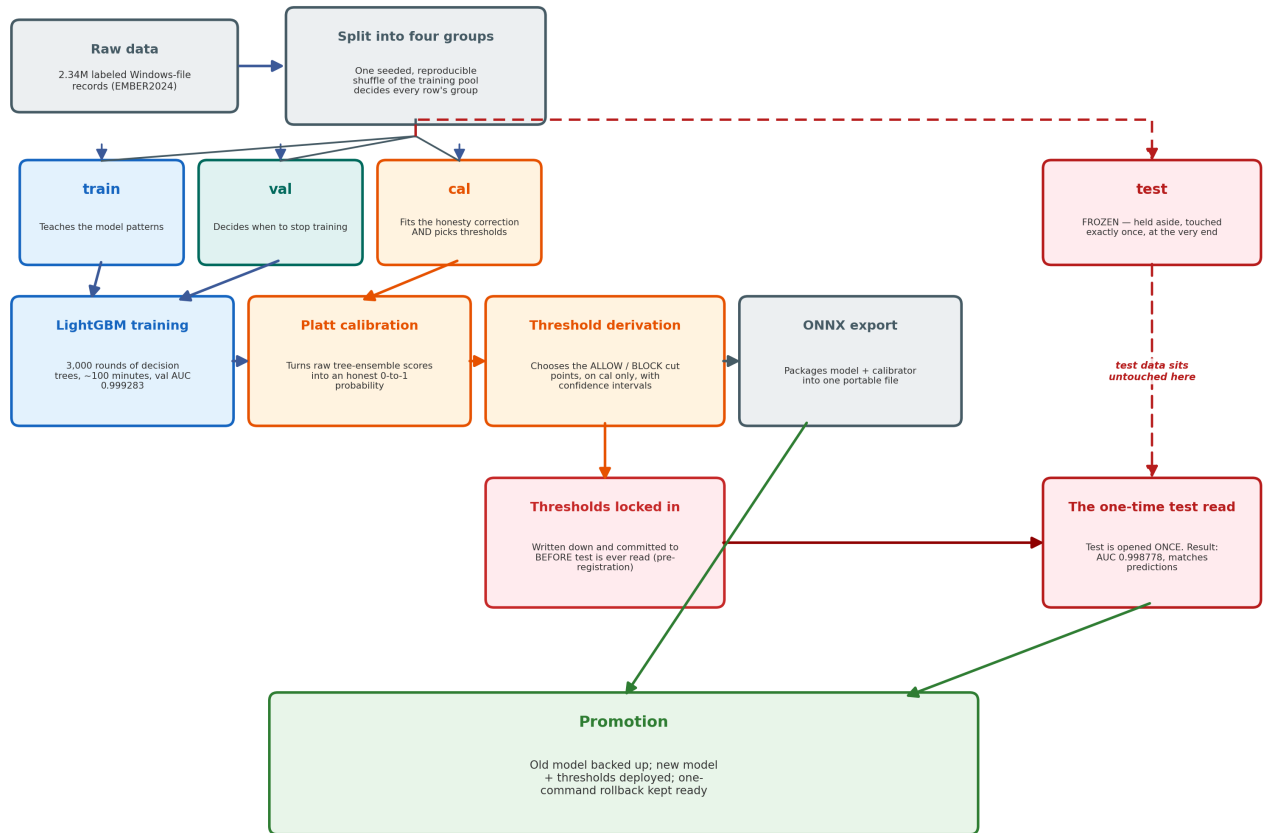


Figure 3. The static model's training pipeline for this retrain.

## 6. Test Results — Old vs New

| Metric                            | Old model (test)                                   | New model (test)        |
|-----------------------------------|----------------------------------------------------|-------------------------|
| Validation AUC                    | 0.999319                                           | 0.999283                |
| Test AUC                          | 0.998841                                           | 0.998778                |
| False-positive rate at ALLOW line | 1.00% (2,699 / 269,940 safe files wrongly flagged) | 1.21% (3,274 / 269,940) |
| Detection rate at ALLOW line      | 98.03%                                             | 98.21%                  |
| False-positive rate at BLOCK line | 0.10% (269 / 269,940)                              | 0.10% (273 / 269,940)   |
| Detection rate at BLOCK line      | 91.68%                                             | 91.49%                  |

A plain-language note on how to read this table: the old model's numbers above look almost suspiciously tidy, especially the false-positive rate at the ALLOW line landing on almost exactly 1.00% — because, as Section 4 explained, those thresholds were originally chosen by looking directly at this same test data. That is not a fair test; it is closer to regrading your own exam with the answer key open. The new model's numbers are close to the old ones, sometimes very slightly worse on paper (1.21% vs 1.00% false positives at the looser ALLOW line) — but they are the **first numbers in this model's history that were never allowed to see the test data until after every decision about them was already locked in**. A slightly less flattering, honest number is worth more than a flattering number that cannot be trusted. The right way to describe this change is "trustworthy for the first time," not "worse."

One more honest caveat, stated directly rather than glossed over: the BLOCK line's false-positive rate on this final exam (0.10%, with a statistical confidence range of 0.0895% to 0.1139%) sits right on top of this project's own long-standing target of 0.1% — not comfortably below it. Section 8 returns to this.

## 7. What's Better Now

- **The crash is fixed, and proven at full scale**, not just patched around. The retraining process that used to reliably run out of memory now completes cleanly, with the specific memory-releasing behavior directly measured and matched almost exactly to the expected arithmetic, both in a small controlled test and at the real 2.34-million-row production scale.
- **The thresholds are honest for the first time.** As explained in Sections 4–6, the cut-lines now in production were derived on data the model never learned from and locked in writing before the one graded exam was ever opened — structurally impossible to quietly adjust after the fact, because the adjustment window had already closed.
- **The calibration correction is no longer running into the "saturation" problem** described in Section 4. That earlier problem came from fitting the correction curve on the model's own already-compressed output; this retrain's correction (like the version already running in production since a prior, separate fix) is fit on the model's raw, uncompressed score, and its resulting shape (a modest, well-behaved adjustment) is consistent with that earlier fix rather than a return to the old, nearly-all-or-nothing compression pattern. In plain terms: the correction curve now behaves like a gentle dial rather than an on/off switch.
- **A full, written record and a tested rollback exist.** Every step above — the crash investigation, the fix, the training run, the threshold menu, the pre-registered prediction, the one-time test read, and the promotion itself — is documented with specific measured numbers, and the previous model can be restored with one command if ever needed.

## 8. What's Still a Problem

Stated with full honesty, not minimized:

- **The BLOCK threshold sits right at its own target line, not comfortably under it.** As noted in Section 6, the measured false-positive rate at the BLOCK line on the one-time test read was 0.101%, with a statistical confidence range that includes this project's own 0.1% target rather than sitting cleanly below it. This is not a failure, but it is not a comfortable margin either, and it was reported exactly as measured rather than rounded favorably.
- **The known real-world false-positive pattern was NOT fixed by this work, and this session confirmed it directly.** Five real Windows files with known, documented identities — including `svchost.exe` (a genuine, digitally signed Windows system file) and `extractor.exe` (a legitimate but PyInstaller-packaged program) — were re-scored with both the old and new model on identical extracted features. All five verdicts were unchanged between old and new, and two of them, `benign_test_50mb.exe` and `extractor.exe`, scored at or above the BLOCK line under *both* models. Every one of these five files' scores also rose slightly under the new model, for reasons that were not separately investigated (most likely explanation: the combined effect of a different set of trees and a different calibration curve, not confirmed by a dedicated test). This retrain fixes how the cut-lines were chosen — it does not, and was never intended to, fix the underlying reason certain legitimate files score high in the first place.
- **The safety cap described in Section 3 stays in place, deliberately, because of the point directly above.** Because a static BLOCK score alone still cannot be fully trusted not to flag a legitimate program, the system continues to require a second, independent signal (from Cortex-Memory or Cortex-Network) before a static BLOCK verdict is allowed to actually stop a file. Lifting that cap requires closing the underlying feature-reading mismatch mentioned in Section 4 and re-validating on real-world files with none of them landing at or above the BLOCK line — none of which happened in this session, by design; it was out of scope.
- **One inactive training setting was found and left alone.** A setting called `subsample` (technically "bagging fraction"), intended to make each round of tree-building look at a random 80% slice of the data for better generalization, was directly confirmed to have no effect at all on the resulting model — a companion setting that controls how often this random sampling actually happens was set to "never," silently disabling it. This is harmless: it does not make the model worse, it simply means one intended technique was not actually contributing anything. It was measured and documented, not changed, since fixing it would itself be a training-behavior change requiring its own separate before/after comparison and sign-off — deliberately out of scope for a session about fixing a crash and a scoring-honesty bug, not about tuning the model further.

## 9. What Changed, Concretely

Four commits landed this session's work, in this order:

- 1 8192452 — *"static: split-discipline retrain + memory-lifecycle fix (train\_from\_holder, calibrate on cal)"*. Rewrote the training script's data-splitting logic to carve out the dedicated calibration slice described in Section 5, restructured how large training arrays are passed between functions to fix the memory-lifecycle bug (Section 5b–c), and added a regression test that directly checks the fix holds. Five files changed (367 lines added, 84 removed overall): `models/static_lgbm.py` (185 lines touched), `scripts/train_static.py` (209 lines touched, substantially restructured), a new 52-line test file, and two small documentation-reference fixes.
- 2 48f55f0 — *"scripts: derive static ALLOW/BLOCK candidate thresholds on the cal split (Clopper-Pearson CIs)"*. Added the new threshold-derivation tool (`scripts/derive_static_thresholds.py`, 277 new lines) described in Section 5e, which reuses the exact same data-loading code the training script uses (so the calibration rows can never silently drift apart between the two), reports statistically rigorous confidence intervals on every candidate threshold, and — by design — never opens the held-out test data at all. Five new tests.
- 3 9030d6a — *"docs(OPEN\_ITEMS): record static split-discipline retrain, pre-registration, one-time test read and promotion; add reports/static\_retrain\_20260921"*. The full written record: the memory-lifecycle root cause and fix, the training run, the threshold menu, the pre-registration document, the test results, the promotion, and the rollback procedure — plus a new `reports/` folder containing the raw evidence files (pre-registration text, test-evaluation output, threshold-derivation output, and a SHA-256 fingerprint manifest tying them all together).
- 4 438d936 — *"config(thresholds): promote static split-discipline retrain (thresholds derived on cal, not test)"*. The actual cut-over: `config/thresholds.yaml` updated with the new ALLOW threshold (0.4789517595186417) and BLOCK threshold (0.9811748406902472), paired to the specific trained model by cryptographic fingerprint, plus updated documentation comments in `ARCHITECTURE.md` and `inference/policy_engine.py` recording exactly how these numbers were derived and what they superseded.

**Threshold values, before and after this retrain:**

|                                                                | Old (superseded)   | New (current)      |
|----------------------------------------------------------------|--------------------|--------------------|
| ALLOW threshold (score below this → ALLOW)                     | 0.5471026402140103 | 0.4789517595186417 |
| BLOCK threshold (score at/above this → BLOCK, capped to ALERT) | 0.9798998555119341 | 0.9811748406902472 |

## CortexPipeline.scan() — Technical Sequence

*inference/pipeline.py, in the exact order the code executes*

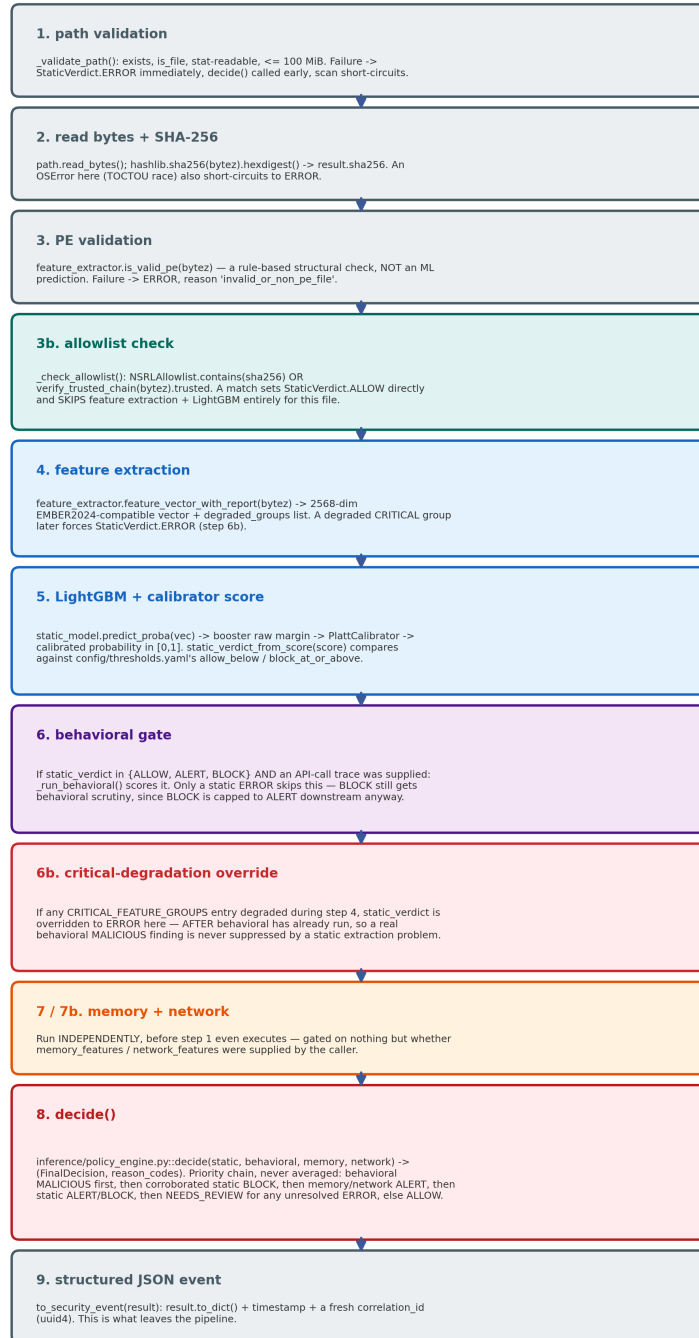


Figure 4. The technical scan sequence in inference/pipeline.py.

## 10. What's Left

- **Handoff to the integration team has not happened yet.** A detailed delivery manifest (file sizes, cryptographic fingerprints, the exact input/output contract of the portable exported model file, and known behavioral quirks) was written and committed to the repository during this session, but actually handing it to and coordinating with the team that integrates this model into the live endpoint agent is separate work that has not started.
- **The feature-parity work is the real prerequisite before the safety cap in Section 3 can even be reconsidered.** That work — building a side-by-side comparison harness against the reference feature-reading implementation, closing the specific measured gaps it finds, retraining again, and then re-validating against real-world files with none landing at or above the BLOCK line — has not been started in this session and was explicitly out of scope for it.
- **Specific files still show outdated numbers and have not been corrected**, because they currently contain unrelated in-progress edits by the project maintainer and touching them risked colliding with that work: `README.md` (its worked example still references a training flag that no longer exists) and `PROJECT_HISTORY_REPORT.md` (still quotes an older, now twice-superseded pair of threshold values). A ready-to-apply patch for the README's example was prepared and verified but deliberately not applied, exactly so it does not collide with the maintainer's own uncommitted changes to that file.

## Closing

In short: a malware detector that had one broken measuring stick and one machine that crashed every time someone tried to fix it. This session found and fixed the actual reason the machine kept crashing — a single stray reference to eighteen gigabytes of data that Python couldn't let go of — and then used the now-working machine to redo the broken measuring stick honestly: picking the safe/dangerous cut-off lines using data the model had never seen, writing those lines down in permanent ink before anyone was allowed to check the final answer, and only then checking it. The new numbers are almost identical to the old ones, which were flattering but untrustworthy; the new ones are slightly less flattering in places and, for the first time, can actually be believed. A separate, already-known problem — certain legitimate large programs occasionally scoring as dangerous — was not touched by this work and remains guarded against by an existing safety brake that this session deliberately left in place rather than removed.